Exercise Sheet 02

The Bonus Run

House rules.

- One file per exercise, in `my_work/`: `ex1_bonus_rule.py`, `ex2_bonus_run.py`, …
- Start each file by typing the **Given** lines. Then write the rest yourself.
- Run it (`python my_work/ex1_bonus_rule.py`) and compare with the **expected output**, character by character.
- Indentation is not decoration in Python: the lines *inside* an `if` or a `for` are the ones indented by 4 spaces.
- Stuck for more than 10 minutes? Cheat sheet first, then skip and come back later.
- Hand in: commit and push to `lesson-02`, then open a pull request. Tell Lars which ticket was the most annoying.

Exercise 1 One bonus decision

10 min · *ex1_bonus_rule.py*

> **From Siobhan (Reward):** *"The bonus rules, as Gerald wrote them on a napkin: nothing under 12 months of service. Otherwise 8 % for a rating of 4 or 5, 4 % for a 3, nothing below that. Can Python read a napkin?"*

GIVEN

```
name = "Priya Patel"
months_of_service = 14
rating = 4
salary = 31000
```

(a) Use `if` / `elif` / `else` to set a `bonus_rate` *and* a short `reason` text. Check the months first.

(b) Print the bonus in pounds and the reason in brackets.

(c) Now change the Given values one at a time: months 6, rating 3, rating 1. Does the reason change the way you expect? Put the values back afterwards.

EXPECTED OUTPUT

```
Priya Patel: bonus £2,480.00 (rating 4 or 5)
```

Hint: Only the **first** condition that is true runs. Order matters: if the rating check came first, a new starter with a 5 would get paid.

Exercise 2 The bonus run

15 min · *ex2_bonus_run.py*

> **From Siobhan:** *"Lovely. Now do it for everyone, please. In Excel I'd drag the formula down. What's the Python for 'drag down'?"*

GIVEN

```
records = "Priya Patel/31000/4;Mo Khan/36500/3;Siobhan Walsh/28000/5;Tom Brown/24500/2;Amara Okafor/41000/4"

for record in records.split(";"):
    name, salary_text, rating_text = record.split("/")
```

The two Given lines at the bottom give you one employee per trip through the loop: `record.split("/")` cuts `"Priya Patel/31000/4"` into three pieces and puts them in three variables. (Everyone has 12+ months this time.)

(a) Inside the loop: convert the texts to numbers, work out the rate (8 % for 4+, 4 % for 3, else 0) and print one line per person: name left-aligned in 14 characters, bonus right-aligned in 10 with two decimals and a thousands separator.

(b) Keep a running `total`. Where must `total = 0` go: inside or before the loop? Try both and watch what happens.

(c) After the loop, print a line of 24 dashes and the total in the same layout.

EXPECTED OUTPUT

```
Priya Patel     2,480.00
Mo Khan         1,460.00
Siobhan Walsh   2,240.00
Tom Brown           0.00
Amara Okafor    3,280.00
------------------------
Total           9,460.00
```

Exercise 3 Twelve months of pension

10 min · `ex3_payslip_lines.py`

> **From Priya (Payroll):** *"Someone's asked how much pension they'll have paid by the end of the year, month by month. MACRO-9000 answered 'Run-time error 6: Overflow', which I think means 'a lot'."*

GIVEN

```
annual_salary = 32500
pension_rate = 0.05
```

(a) Work out the monthly pension (5 % of one twelfth of the salary).

(b) Loop over the months with `range(1, 13)`. Each time, add the pension to a running total and print the line. Month numbers are right-aligned in 2 characters.

(c) After the loop, print the yearly total.

EXPECTED OUTPUT

```
Month  1: pension £135.42  total so far £135.42
Month  2: pension £135.42  total so far £270.83
Month  3: pension £135.42  total so far £406.25
Month  4: pension £135.42  total so far £541.67
Month  5: pension £135.42  total so far £677.08
Month  6: pension £135.42  total so far £812.50
Month  7: pension £135.42  total so far £947.92
Month  8: pension £135.42  total so far £1,083.33
Month  9: pension £135.42  total so far £1,218.75
Month 10: pension £135.42  total so far £1,354.17
Month 11: pension £135.42  total so far £1,489.58
Month 12: pension £135.42  total so far £1,625.00
Pension paid this year: £1,625.00
```

Hint: `range(1, 13)` gives 1 up to **and not including** 13. Same rule as slicing.

Exercise 4 Salary bands

10 min · `ex4_salary_bands.py`

> **From Gerald (MD), Friday 16:52:** *"How many people are in each salary band? A is under 25k, B under 32k, C under 40k, D is the rest. And what's the top salary? Don't tell me who. Actually do. No, don't. G."*

GIVEN

```
salaries = "24500,28000,31000,36500,41000,52000,19500,33000"
```

(a) Loop over the salaries (`salaries.split(",")` gives you one text at a time; convert it). Print each salary and its band.

(b) Count how many land in each band with four counters, and track the highest salary seen so far.

(c) After the loop, print the counts and the highest salary.

EXPECTED OUTPUT

```
£24,500 -> band A
£28,000 -> band B
£31,000 -> band B
£36,500 -> band C
£41,000 -> band D
£52,000 -> band D
£19,500 -> band A
£33,000 -> band C
A: 2, B: 2, C: 2, D: 2
Highest salary: £52,000
```

Hint: Because `elif` only runs when the earlier checks were false, "under 32k" doesn't need to say "and 25k or more". Python already knows.

Exercise 5 How many years?

10 min · `ex5_years_to_target.py`

> **From Mo (IT):** *"Hypothetically. If someone on 28k got 3 % a year, how many years until 35k? Asking for a friend. The friend is me."*

GIVEN

```
salary = 28000
target = 35000
raise_percent = 3
```

(a) You don't know how many times to loop, so use `while salary < target`. Each time: apply the raise, count the year, print it.

(b) After the loop, print how many years it took.

(c) Set `raise_percent = 0` and run it. Press `Ctrl+C` when you've seen enough. Then put the 3 back. Congratulations, you've met the infinite loop.

EXPECTED OUTPUT

```
Year 1: £28,840.00
Year 2: £29,705.20
Year 3: £30,596.36
Year 4: £31,514.25
Year 5: £32,459.67
Year 6: £33,433.46
Year 7: £34,436.47
Year 8: £35,469.56
Target reached after 8 years.
```

Exercise 6 Holiday allowance

10 min · `ex6_holiday.py`

> **From Priya:** *"Holiday is 25 days, plus 1 after 2 years, plus 3 after 5, plus 5 after 10 (not added up: the best one applies). The export has 'left' for leavers. Skip those, Dave's version gave them 30 days and a leaving card."*

GIVEN

```
this_year = 2025
starts = "2024,2021,left,2019,2015,left,2009"
```

(a) Loop over the entries. If an entry is `"left"`, use `continue` to jump to the next one.

(b) Otherwise work out the years of service and the days (`days += 5` is short for `days = days + 5`), and print the line.

(c) Make it say "1 year" rather than "1 years". Small things matter on a payslip.

EXPECTED OUTPUT

```
Started 2024: 1 year of service -> 25 days holiday
Started 2021: 4 years of service -> 26 days holiday
Started 2019: 6 years of service -> 28 days holiday
Started 2015: 10 years of service -> 30 days holiday
Started 2009: 16 years of service -> 30 days holiday
```

Exercise 7 Dave's payroll calendar STRETCH

15 min · `ex7_payroll_calendar.py`

> **From Priya:** *"Dave's calendar macro: timesheets every 3rd day, pension every 5th, payday on the 15th and 30th (which is both, so it says both). Days with nothing on are left out. It was the only thing of his that worked. Then it stopped."*

GIVEN

```
for day in range(1, 31):
```

(a) Inside the loop decide the label. Think about the **order** of the checks: day 15 must say "PAYDAY + pension", not "pension". Use `continue` for days with nothing on.

(b) Then, with a `while` loop that counts up from 21, find the first payday after day 20 and print it.

EXPECTED OUTPUT

```
Day  3: timesheets
Day  5: pension
Day  6: timesheets
Day  9: timesheets
Day 10: pension
Day 12: timesheets
Day 15: PAYDAY + pension
Day 18: timesheets
Day 20: pension
Day 21: timesheets
Day 24: timesheets
Day 25: pension
Day 27: timesheets
Day 30: PAYDAY + pension
First payday after day 20: day 30
```

Hint: `day % 15 == 0` is true on the 15th and 30th. Programmers call this exercise FizzBuzz. Now you know the secret handshake.

Bug hunt Dave's Leftovers #2

15 min · `bughunt/bonus_bug.py`

Dave's 2019 bonus run. When Siobhan ran it, *everyone* got a bonus, including people who'd been there a week, and the total came to minus forty pounds. There are **three** bugs: one crashes, one pays the wrong people, one wrecks the total even when every line is right. The rules are in the file header.

EXPECTED OUTPUT ONCE FIXED

```
Priya Patel     2,480.00
Tom Brown           0.00
Mo Khan         1,460.00
Biscuit             0.00
------------------------
Total           3,940.00
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.